

BSc and MSc Student Project Portfolio

Graph applications, self-supervised time series, modern Graph ML and vision foundation models

How to use this portfolio

The portfolio contains sixteen projects in four categories. Each entry states the task, public dataset, starting papers and code, required baselines, evaluation protocol, Bachelor scope and possible Master contribution.

Bachelor projects focus on implementation, comparison and one focused ablation. **Master projects** add a stated research question, a method or evaluation change, multiple runs and a detailed analysis. A controlled negative result is acceptable when the evaluation answers the stated question.

Department of Electrical and Computer Engineering
Aarhus University

Supervisor: Naveed

Project scope and evaluation

What counts as a BSc or MSc contribution

Dimension	Bachelor project	Master project
Starting point	One dataset and one established method	Verified anchor-paper reproduction plus required baselines
Contribution	Documented pipeline, fair comparison, one focused ablation	New mechanism, evaluation protocol, graph/view construction or transfer/robustness study
Evidence	Correct split, two or three baselines, error analysis	Multiple seeds, uncertainty, matched budgets, ablations and resource reporting where relevant
Outcome	Explain whether and when the method helps	Answer the stated research question; a controlled negative result is valid

Portfolio index

Compute estimates assume compact models, moderate batch sizes and no large hyperparameter sweep. Projects marked as requiring a preliminary check need a data or licence review before work begins.

ID	Project	Suggested level	Compute
1.1	Remaining useful life prediction for aircraft engines	Both	Very light to light
1.2	Multi-horizon traffic forecasting with data-driven graph structure	Both	Light to moderate
1.3	Illicit Bitcoin transaction detection	Both	Moderate
1.4	Graph-based movie recommendation with accuracy and coverage	BSc preferred	Very light
1.5	Molecular HIV activity prediction with calibrated GNNs	Both	Light to moderate
2.1	Contrastive learning for wearable activity recognition	Both	Light
2.2	Lead-aware masked learning for 12-lead ECG	MSc preferred	Moderate
2.3	Self-supervised forecasting representations for energy series	Both	Light to moderate
2.4	Self-supervised learning on heat-pump smart-meter data	Preliminary check	Light to moderate after preprocessing
2.5	Graph self-supervised learning from vibration signals	MSc preferred	Moderate preprocessing
3.1	Inductive sensor embeddings for STAEformer	MSc preferred	Moderate
3.2	Inductive evaluation of GRACE	Both	Very light
3.3	Structure-aware augmentations for graph contrastive learning	Both	Light to moderate
3.4	Compute-aware routing between GraphGPS branches	MSc preferred	Light to moderate
3.5	Question-adaptive retrieval budgets for G-Retriever	MSc optional	Moderate
4.1	Adapting DINO backbones for land cover or industrial anomalies	Both	Moderate

How to choose and run a project

A simple decision process

1. Choose the **task**, not the fashionable model: node classification, link prediction, graph classification/regression, temporal forecasting or representation learning.
2. Download the data and run one end-to-end baseline on a small subset before the formal project begins.
3. Write a one-sentence research question and identify the comparison that would falsify the proposed explanation.
4. Freeze the split, primary metric and compute budget before extensive model development.
5. Treat the authors' repository as an executable specification to inspect and verify—not as the thesis contribution.

Shared protocol for self-supervised and pretrained projects

Projects 2.1–2.5, 3.2, 3.3 and 4.1 must report the following settings whenever they are meaningful:

- **Frozen linear evaluation:** pretrain without labels, freeze the encoder and train only a linear or otherwise explicitly lightweight head.
- **Fine-tuning:** initialise from the pretrained encoder and update it with labels.
- **Fully supervised baseline:** train the same encoder architecture from scratch using labels.
- **Random-initialisation control:** freeze a randomly initialised encoder and train the same head. Without this, the value of pretraining is ambiguous.
- **Label-efficiency curve:** report several budgets, typically 1%, 5%, 10% and 100%, rather than one full-label result.

Leakage rule

Group splits by the real independent unit: subject, patient, household, engine or bearing—never by window. On graphs, state whether pretraining is transductive (evaluation nodes/graphs visible without labels) or strict inductive (held out entirely). On time series, use chronological splits where the task is forecasting.

Report ten seeds for small citation networks and at least three elsewhere. Report results even when they do not beat a leaderboard; the evaluation should answer the stated question.

Licences and access

Resource	Licence or status	Practical consequence
UCI HAR	CC BY 4.0	Open with attribution
PTB-XL	CC BY 4.0	No credentialing required
Paderborn KAT	CC BY-NC 4.0	Non-commercial academic use
MVTec AD	CC BY-NC-SA 4.0	Non-commercial; share-alike
DINOv2	Apache 2.0	Ungated
DINOv3 weights	Custom Meta licence	Gated; non-standard; review for industry use
HEAPO data	Not displayed in Zenodo Rights field	Confirm before starting; code is MIT, paper is CC BY 4.0

Category 1 GNN applications for real-world data

These projects begin with a practical decision: estimate remaining life, forecast traffic, detect illicit activity, recommend an item or screen a molecule. The student must explain what the nodes and edges mean and test whether the graph adds value beyond a non-graph model.

Application project requirements

Specify a public dataset, graph construction, task type, primary metric, non-graph baseline and graph baseline. Include an edge-removal, edge-randomisation or alternative-graph experiment, and discuss leakage and limitations.

The evaluation should show whether sensor dependence, transaction flow, road connectivity, user–item structure or molecular bonding changes the prediction.

Learn first and find ideas

- [Stanford CS224W course and project guidance](#) and the [CS224W student-project archive](#).
- [PyTorch Geometric dataset catalogue](#), [Open Graph Benchmark](#), [SNAP datasets](#) and the [Temporal Graph Benchmark](#).
- Stanford examples: [traffic as a dynamic graph](#), [water quality with RGCNs](#), [fake-news detection](#) and [NFT transaction prediction](#).

Project 1.1 Remaining useful life prediction for aircraft engines

Task	Graph-level temporal regression
Recommended level	BSc or MSc
Compute	Very light to light; CPU or 4 GB GPU
Starting paper	Huang, He and Sick, Spatio-Temporal Attention GNN for RUL , 2024

What you will do. Predict how many operating cycles remain before a turbofan engine fails. Each sensor channel becomes a node; edges express statistical or learned relationships among sensors. A temporal encoder then follows how the sensor graph changes as an engine ages.

Project goal. Given a recent window of readings from one engine, predict one remaining-life value. The scientific question is whether explicitly modelling sensor interactions improves over treating the measurements as an ordinary multivariate sequence.

Data. Use [NASA C-MAPSS](#). Start with FD001: 100 training engines, 100 test engines, three operating-condition variables and 21 sensor channels. FD002–FD004 are optional transfer sets. Split by **engine**, never by windows from the same engine.

Methods and starting code. Linear or MLP regression, GRU/LSTM/TCN, GCN–GRU, GAT–GRU and the anchor spatiotemporal attention model. Compare correlation, random and learned sensor graphs. [PHMGNNBenchmark](#) and the [GNN RUL benchmarking repository](#) are useful code references; students should still document their own preprocessing.

Required baselines. A naive constant or degradation-stage baseline, one graph-free sequence model and one compact spatiotemporal GNN. The graph experiment must include a no-edge or random-edge control.

Evaluation. MAE or RMSE is primary; the asymmetric NASA score is optional. Report error by early, middle and late life. Fit normalization and graph construction on training engines only and report at least three seeds for MSc work.

Bachelor scope. Use FD001. Compare a GRU with one compact GNN, one graph construction and one random/no-edge ablation. Submit a documented pipeline and state whether the graph changes performance.

Master research question and contribution. Research question: **Can a sparse, time-varying sensor graph improve RUL accuracy and robustness when sensors become noisy or unavailable?** Develop one graph-learning mechanism, compare it with fixed correlation and random graphs, and include missing-sensor, uncertainty or cross-subset transfer analysis. Merely rerunning an available implementation is not the contribution.

Practical notes. FD001 is easy to run. The main risks are window leakage, inconsistent RUL capping and comparing results produced with different normalization or scoring rules. Freeze these choices before modelling.

Project 1.2 Multi-horizon traffic forecasting with data-driven graph structure

Task	Node-level temporal regression
Recommended level	BSc or MSc
Compute	Light to moderate; 4–8 GB GPU
Starting paper	Li et al., DCRNN , and Wu et al., Graph WaveNet

What you will do. Forecast traffic speed at every road sensor 15, 30 and 60 minutes ahead. The supplied road graph is a useful starting point, but congestion can couple sensors in ways that do not follow the shortest road distance.

Project goal. Determine whether a road-distance graph, a training-only correlation graph or a learned graph gives the most reliable forecasts, especially at the 60-minute horizon.

Data. Use **METR-LA** from the [official DCRNN repository](#); 207 sensors and 34,272 five-minute observations. [PEMS-BAY](#) with 325 sensors is the optional transfer target. Use the established chronological 70/10/20 split.

Methods and starting code. Historical average, linear regression, GRU, DCRNN, STGCN, [Graph WaveNet code](#), MTGNN and a compact Transformer. A dynamic graph should be sparse or low-rank so its cost is explicit.

Required baselines. Historical average, graph-free GRU/Transformer and one fixed-graph spatiotemporal GNN. Compare supplied distance adjacency, training-only correlation adjacency, a single learned static adjacency and the proposed variant.

Evaluation. MAE is primary; report RMSE and MAPE at 15/30/60 minutes. Fit scalars and correlation graphs on the training period only. For dynamic adjacency, report memory, latency and edge stability as well as forecast error.

Bachelor scope. Compare historical average, GRU and one spatiotemporal GNN on the supplied graph. Add one no-edge or randomized-edge control.

Master research question and contribution. Research question: **Does a time-of-day and traffic-regime-conditioned sparse adjacency improve long-horizon forecasting over both the road graph and a single learned static graph?** Match depth and width, use at least three seeds, and examine whether rush-hour edges differ from free-flow edges and whether learned edges correspond to plausible topology. A robustness-and-transfer route using missing sensors and METR-LA to PEMS-BAY transfer is an alternative, but it must be narrowed to one preregistered question.

Practical notes. Per-timestep dense adjacency is unnecessarily expensive and unstable. Profile early and enforce top-k sparsity or low rank. This project and Project 3.1 both use traffic data, but this one studies the graph itself; Project 3.1 studies unseen-sensor representations.

Project 1.3 Illicit Bitcoin transaction detection

Task	Node classification on a temporal graph
Recommended level	BSc or MSc
Compute	Moderate; sparse mini-batches on 4–8 GB GPU
Starting paper	Weber et al., Anti-Money Laundering in Bitcoin

What you will do. Each Bitcoin transaction is a node, and directed payment flows are edges. The model predicts whether a labelled transaction is licit or illicit while respecting the order in which transactions occurred.

Project goal. Test whether neighbourhood information helps detect rare illicit transactions beyond the transaction's own features, without allowing information from future time steps to leak into training.

Data. Use the [PyG Elliptic Bitcoin Temporal loader](#): 203,769 transaction nodes, 234,355 edges, 165 features and 49 time steps. Unknown labels are **unlabelled**, not negatives.

Methods and starting code. Logistic regression or gradient boosting on node features; GCN, GraphSAGE or GAT; EvolveGCN or TGN for an MSc extension. Use class weighting or focal loss only after establishing an unweighted baseline. PyG's loader is the recommended data starting point.

Required baselines. Feature-only classifier, static GCN/GraphSAGE and a no-edge or random-edge control. A temporal model is required only for the MSc temporal route.

Evaluation. Chronological splits only. Use AUPRC or illicit-class F1 as the primary metric and report illicit recall at a fixed false-positive rate. Exclude unknown nodes from supervised loss and labelled-node metrics; examine performance over time.

Bachelor scope. Compare a feature-only model with GCN or GraphSAGE and analyse how edge removal changes performance. Include class-imbalance and per-time-step error analysis.

Master research question and contribution. Research question: **Can a temporally calibrated GNN remain useful under concept drift and missing transaction edges?** Add one temporal mechanism or uncertainty method, test edge deletion and temporal shift, and connect threshold choices to investigator workload. A negative result may show that features dominate topology during particular regimes.

Practical notes. Random splitting makes this task unrealistically easy. Keep time ordering, labels and threshold selection fixed across methods. Mini-batching or snapshot processing keeps the graph manageable.

Project 1.4 Graph-based movie recommendation with accuracy and coverage

Task	Link prediction and ranking on a bipartite graph
Recommended level	BSc recommended; limited MSc scope
Compute	Very light; CPU or low-memory GPU
Starting paper	He et al., LightGCN , SIGIR 2020

What you will do. Represent users and movies as two node types. A rating or interaction is an edge. The model scores unseen user–movie pairs and recommends a ranked list, while also checking whether recommendations collapse onto popular titles.

Project goal. Measure whether message passing improves top-k ranking over popularity and matrix factorization and whether any gain comes at the cost of catalogue coverage.

Data. Use [MovieLens 100K](#): 100,000 timestamped ratings from about 1,000 users on 1,700 movies, roughly a 5 MB download.

Methods and starting code. Popularity, item/user nearest neighbours, matrix factorization and LightGCN. The [official LightGCN repository](#) is a reference, but its data protocol must be adapted transparently to MovieLens 100K.

Required baselines. Popularity and matrix factorization are mandatory; LightGCN is the graph model. Use identical candidate sets and negative-sampling rules.

Evaluation. Use a per-user chronological split and remove validation/test interactions from the training graph. NDCG@20 or Recall@20 is primary; also report catalogue coverage and average popularity of recommended items.

Bachelor scope. Compare popularity, matrix factorization and LightGCN. Study depth or negative sampling and report both accuracy and coverage.

Master research question and contribution. For an MSc project, use a narrower research question such as: **Can a calibrated diversity objective improve long-tail exposure without materially reducing NDCG?** This requires user-activity and item-popularity subgroup analysis, multiple seeds and a defined Pareto trade-off. Without this extension, the project is suitable for Bachelor scope.

Practical notes. The dataset and baseline setup are small. The principal pitfalls are leakage from future interactions and evaluation on sampled negatives that differ across methods.

Project 1.5 Molecular HIV activity prediction with calibrated GNNs

Task	Graph classification
Recommended level	BSc or MSc
Compute	Light to moderate; 4–8 GB GPU
Starting paper	Hu et al., Open Graph Benchmark

What you will do. A molecule is a graph: atoms are nodes and bonds are edges. Predict whether the complete molecule is active in the HIV benchmark assay and test whether confidence estimates remain useful on molecular scaffolds not seen during training.

Project goal. Compare learned graph representations with established chemical fingerprints under the official out-of-scaffold split.

Data. Use [OGB ogbg-molhiv](#): 41,127 molecular graphs, official scaffold split and official ROC-AUC evaluator. The [OGB loader guide](#) handles download and split indices.

Methods and starting code. Morgan fingerprints with logistic regression or random forest; GCN; GIN/GINE; PNA; compact GraphGPS. OGB provides baseline code, and [PyG](#) provides the layers and loader.

Required baselines. Fingerprint classifier, GCN and GIN/GINE. Bond-feature and pooling ablations are required. If self-supervised pretraining is added, follow the shared SSL protocol.

Evaluation. Use only the official scaffold split and evaluator. ROC-AUC is primary; AUPRC, Brier score or expected calibration error are supporting metrics. Report at least three seeds. **Do not compare scaffold-split results with random-split figures.**

Bachelor scope. Compare fingerprints, GCN and GIN/GINE and inspect errors on unfamiliar scaffolds. Include one bond-feature or pooling ablation.

Master research question and contribution. Research question: **Can uncertainty-aware training improve calibration on held-out scaffolds without sacrificing discrimination?** Compare temperature scaling, deep ensembles or a compact evidential/uncertainty variant on identical predictions and report both performance and calibration by scaffold novelty. A self-supervised route is possible, but it must be distinct from Project 3.3.

Practical notes. The dataset is well established and manageable. The main danger is silently changing the split or selecting a threshold on test data.

Category 2 Self supervised representation learning for time series

Industrial and healthcare time series are abundant, but labels often require a technician, clinician or field inspection. Self-supervised learning creates a training signal from the recordings themselves, then tests whether a small downstream head can solve a labelled task with less supervision.

Three ideas to understand

Contrastive learning makes representations of two valid views of the same recording similar. **Masked modelling** reconstructs hidden samples or patches. **Latent prediction or JEPA-style learning** predicts hidden representations rather than raw values and therefore needs explicit collapse monitoring.

Possible MSc contributions include physically valid view or mask construction: rotating inertial axes consistently, hiding related ECG leads, aligning energy windows by seasonal phase, matching smart-meter views by outdoor temperature or constructing vibration graphs from bearing kinematics.

Learn first and find data

- Introductory methods with code: [TS-TCC](#), [TS2Vec](#), [TF-C](#) and [SimMTM](#).
- Dataset sources: [UCI Machine Learning Repository](#), [PhysioNet](#), [UCR/UEA time-series archive](#), [Monash forecasting archive](#) and [Zenodo](#).
- Stanford CS224W is graph-focused, so its project archive is most directly useful for Categories 1 and 3. For this category, use the method repositories above and the shared evaluation protocol in this document.

Collapse warning

A latent-predictive model can minimise its objective with nearly constant embeddings. Monitor per-dimension variance, covariance or off-diagonal correlation and downstream random-control performance from the first epoch. A smooth pretraining loss does not establish that the representation is useful.

Project 2.1 Contrastive learning for wearable activity recognition

Task	Self-supervised time-series representation learning then classification
Recommended level	BSc or MSc
Compute	Light; CPU or 4 GB GPU
Starting paper	Eldele et al., TS-TCC , IJCAI 2021

What you will do. A phone records multi-channel acceleration and angular velocity while a person walks, sits or climbs stairs. Learn an encoder from the unlabelled sensor windows, freeze it and ask whether a small classifier can recognise activities with few labels.

Project goal. Test whether self-supervised representations approach a fully supervised classifier at 1%, 5%, 10% and 100% label budgets and whether physically plausible sensor views improve orientation robustness.

Data. Use [UCI Human Activity Recognition](#): 30 subjects, six activities, 10,299 windows, 50 Hz inertial data and a subject-independent standard split. The download is about 58 MB and is CC BY 4.0.

Methods and starting code. TS-TCC with the [official code](#), TS2Vec with [official code](#), and optionally [CA-TCC](#). Use the same 1-D CNN or small Transformer encoder in SSL and supervised comparisons.

Required baselines. Random frozen encoder, SSL frozen linear probe, SSL fine-tuning and fully supervised end-to-end training. A simple feature-based classifier is a useful sanity check.

Evaluation. Accuracy and macro-F1 on subject-independent splits, label-efficiency curves and at least three seeds. For robustness, jointly rotate accelerometer and gyroscope triads at test time and measure degradation.

Bachelor scope. Reproduce TS-TCC or TS2Vec, compare frozen and fine-tuned representations with supervised training and produce the full label-efficiency curve.

Master research question and contribution. Research question: **Do physics-consistent SO(3) orientation views improve label efficiency and robustness more than signal-agnostic jitter, scaling and permutation?** Separate gravity from body acceleration, rotate coupled sensor triads consistently, and ablate each step. The insight comes from physically valid view construction, not tuning augmentation strength.

Practical notes. The dataset is small and includes a standard subject-independent split. Preserve subject grouping; never split windows from the same subject across train and test.

Project 2.2 Lead-aware masked learning for 12-lead ECG

Task	Self-supervised multivariate time-series learning then multi-label classification
Recommended level	MSc preferred; BSc comparator variant
Compute	Moderate; 8 GB GPU recommended
Starting paper	Dong et al., SimMTM , NeurIPS 2023

What you will do. A 12-lead ECG observes the same cardiac event from different electrical viewpoints. Instead of masking arbitrary time patches, the model hides complete leads or anatomical lead groups and predicts their latent representations from the visible leads.

Project goal. Compare contrastive and masked/latent-predictive pretraining under low-label conditions and determine whether lead-aware masking learns cross-lead physiology rather than local interpolation.

Data. Use [PTB-XL](#): 21,799 ten-second ECGs from 18,869 patients, available at 100 and 500 Hz under CC BY 4.0 without credentialing. Use folds 1–8 for training, 9 for validation and 10 for testing; these are patient-aware.

Methods and starting code. Begin with [SimMTM official code](#), [TF-C](#) with [official code](#), or TS2Vec. A 1-D ResNet or compact Transformer is the shared encoder. JEPA-style work is an MSc extension and must be described as based on recent preprints rather than an established benchmark.

Required baselines. Random frozen encoder, frozen probe, fine-tuning and the same encoder fully supervised. Compare random temporal masking with complete-lead and anatomical-group masking.

Evaluation. Macro-AUROC on fold 10 for the five diagnostic superclasses is the recommended primary endpoint; add macro-AUPRC for imbalance. Report 1/10/100% label budgets, multiple seeds and patient-level splits. Monitor embedding variance and covariance during latent-predictive training.

Bachelor scope. Reproduce one stable masked method such as SimMTM against TS2Vec and supervised training on the five superclasses. JEPA is not required.

Master research question and contribution. Research question: **Can a lead-identity-conditioned predictor reconstruct the latent representation of missing leads without collapse, and does that improve low-label diagnosis?** Develop lead/group masking, predictor conditioning and collapse diagnostics; ablate masking granularity, EMA schedule and predictor depth. Cross-dataset transfer is optional only after the core study works.

Practical notes. This is methodologically risky because structured masking can collapse. Start at 100 Hz and five superclasses. Keep patient grouping and class definitions identical across methods.

Project 2.3 Self-supervised forecasting representations for energy series

Task	Self-supervised time-series learning then temporal regression
Recommended level	BSc or MSc
Compute	Light to moderate; 4–8 GB GPU
Starting paper	Woo et al., CoST , ICLR 2022

What you will do. Pretrain an encoder on unlabelled energy or transformer measurements, freeze it and fit a small forecasting head. The core comparison asks whether a reusable representation competes with a purpose-built supervised forecaster when labels or trainable parameters are restricted.

Project goal. Evaluate representation quality across several forecast horizons and determine whether season/trend-aware positive pairs help under low-label or transfer settings.

Data. Primary benchmark: **ETT** from the [official ETDataset repository](#), using ETTh1/ETTh2 or ETTm1/ETTm2 with chronological splits. Optional external-validity set: [UCI household power consumption](#), about 2.1 million minute readings from **one household**. It supports chronological forecasting, not household-level classification or cross-household claims.

Methods and starting code. CoST, [TS2Vec](#) and [SimMTM](#). Use DLinear and [PatchTST](#) as supervised reference models. A ridge or linear head on frozen representations is the key diagnostic.

Required baselines. Seasonal-naive forecast, DLinear, PatchTST, random frozen encoder, frozen SSL head and SSL fine-tuning. Use identical input lengths and chronological splits.

Evaluation. MSE and MAE at agreed horizons such as 24/48/168/336/720 where the dataset supports them. Report label/data-efficiency curves, three seeds and training cost. Beating PatchTST is not a project requirement; analyse why the methods differ.

Bachelor scope. Reproduce CoST or TS2Vec on one ETT subset and compare a frozen linear head with DLinear/PatchTST. Add one second horizon or dataset only after the pipeline is stable.

Master research question and contribution. Research question: **Do decomposition-aligned positive pairs improve transfer and low-data forecasting when seasonal phase drifts?** Use a stable period or seasonal decomposition to select positives, compare it with random cropping and frequency-domain contrast, and analyse failure on non-stationary intervals. A negative result is useful if it reveals when estimated seasonality is unreliable.

Practical notes. Period estimation can become the whole thesis. Validate on clean ETT seasonality before using irregular household behaviour. Fit decomposition and normalization on training time only.

Project 2.4 Self-supervised learning on heat-pump smart-meter data

Task	Time-series representation learning then household-level classification
Recommended level	BSc or MSc, subject to a licence and data check
Compute	Light to moderate after preprocessing
Starting paper	Brudermueller et al., HEAPO dataset paper , 2025

What you will do. Pretrain on a large pool of unlabelled household electricity streams, then use a small labelled subset to classify heat-pump type or operational faults. Candidate extensions can define views that account for weather and thermostatic cycling.

Project goal. Determine whether pretraining on unlabelled control households improves label efficiency on the 214 treatment households with smart-meter data and inspection information.

Data. Use [HEAPO on Zenodo](#), with 1,408 households, and the [official MIT-licensed code/dataloader](#). The paper reports 214 treatment and 1,194 control households plus 410 inspection protocols. The Zenodo Rights metadata does not display a dataset licence as of 16 September 2026; the paper's CC BY 4.0 licence does not automatically license the data. Confirm reuse terms with the depositor before starting the project.

Methods and starting code. TS2Vec, TS-TCC and SimMTM or another masked model, optionally weather-conditioned. Use a 1-D CNN, InceptionTime or LSTM as the supervised model. Start with heat-pump type or a sufficiently prevalent protocol label.

Required baselines. Random frozen encoder, frozen SSL probe, SSL fine-tuning and end-to-end supervised training on identical grouped splits. Weather-only and metadata-only baselines help determine what the load series adds.

Evaluation. Group by household. Use balanced accuracy and macro-F1 for heat-pump type; macro-F1 and per-label AUROC/AUPRC for faults. Plot 1/5/10/100% label budgets. Disaggregation is a stretch only after counting households with a dedicated heat-pump meter; use nMAE/nRMSE if the subset is adequate.

Bachelor scope. After the licence check, choose one classification task, one SSL encoder and one supervised baseline. The deliverable is grouped evaluation and a label-efficiency curve.

Master research question and contribution. Research question: **Do outdoor-temperature-matched positives and whole-cycle masking learn heat-pump behaviour more effectively than generic jitter and random masking?** Implement one or both physics-aware views, ablate weather conditioning and compare across household groups. Disaggregation should not be the core question unless the submetered cohort is large enough.

Practical notes. Before starting, verify the dataset licence, field definitions, label prevalence, missingness and the number of households with a dedicated heat-pump meter. If reuse terms cannot be clarified, select another dataset.

Project 2.5 Graph self-supervised learning from vibration signals

Task	Graph-level classification with cross-condition transfer
Recommended level	MSc preferred; BSc supervised variant
Compute	Moderate preprocessing; small models on 4–8 GB GPU
Starting paper	Hou et al., GraphMAE , KDD 2022

What you will do. Convert bearing vibration windows into graphs, pretrain a graph encoder without fault labels and test whether the representation transfers across load or speed conditions better than a raw-signal CNN.

Project goal. Separate two effects: whether the graph construction is useful and whether self-supervision adds value once that graph is fixed.

Data. Reproduction set: [CWRU Bearing Data Center](#). MSc set: [Paderborn KAt](#) with 32 bearings, four operating conditions and 64 kHz vibration plus speed/torque/load channels; [downloads](#). Paderborn is about 5 GB and is documented as CC BY-NC 4.0 for non-commercial academic use in the source publication.

Methods and starting code. GraphMAE or [BGRL](#) on window graphs, a supervised GNN on the same graphs and a 1-D CNN on raw vibration. Use order tracking, envelope spectra and coherence for the MSc physics graph.

Required baselines. Raw-signal 1-D CNN, supervised GNN on the exact same graph, random frozen graph encoder and SSL frozen/fine-tuned encoders. Compare physics-grounded edges with naive similarity and random edges.

Evaluation. Accuracy and macro-F1. Report in-distribution performance, but make cross-operating-condition transfer decisive: train on one condition and test on another using bearing-level grouping and identical held-out conditions for every method.

Bachelor scope. Use CWRU to build a supervised GNN fault classifier with a simple similarity graph, compare it with a 1-D CNN and randomize the edges once.

Master research question and contribution. Research question: **Does a graph built from order-tracked fault-frequency sub-bands and spectral coherence transfer better across operating conditions than an ad hoc similarity graph?**

Compute BPFO/BPFI/BSF from bearing geometry and shaft speed, define a defensible thresholding rule, then evaluate supervised and SSL encoders. The signal-processing graph is the contribution; GraphMAE is a baseline.

Practical notes. This project requires substantial signal preprocessing. Confirm the 5 GB Paderborn download before work begins. Split by bearing, not by windows; otherwise neighbouring samples from the same physical bearing will leak across sets.

Category 3 Implementing modern Graph ML research

These projects begin from an anchor paper with public code and a public benchmark. The first milestone is to reproduce the central mechanism. The Master project adds and tests a new mechanism or a stronger evaluation protocol.

From repository to research question

Installing code, reproducing a table, tuning hyperparameters or moving an unchanged model to another dataset is a starting point, not an MSc contribution. The proposed contributions in this category concern sensor representations, induction protocols, graph augmentations, compute routing and retrieval budgets.

For graph self-supervision, compare the pretrained encoder with a randomly initialised frozen encoder and the same architecture trained fully supervised.

Reading map and Stanford examples

- Spatiotemporal graphs: [survey](#), [DCRNN](#), [STAEformer](#). Dynamic graphs: [TGN](#) and [DyGFormer](#).
- Graph Transformers: [Graphormer](#), [GraphGPS](#). Self-supervision: [DGI](#), [GRACE](#), [GraphCL](#), [GraphMAE](#). Graph RAG: [G-Retriever](#).
- Stanford introductions: [Self-Supervised Learning for Graphs](#), [contrastive molecular pretraining](#), [RingSAGE](#) and [Graph Transformers for TSP](#).

Project 3.1 Inductive sensor embeddings for STAEformer

Task	Node-level multi-horizon regression
Recommended level	MSc preferred; BSc reproduction variant
Compute	Moderate; 4–8 GB GPU
Starting paper	Liu et al., STAEformer , CIKM 2023

What you will do. STAEformer learns a separate adaptive representation for each sensor and input time position. The sensor-specific representation cannot directly handle a sensor that was absent during training. This project develops an embedding generated from the new sensor's graph context and short calibration history.

Project goal. Preserve ordinary forecasting accuracy while improving forecasts for sensors deliberately held out of training.

Data. Use **PEMS04**, supplied by the [official STAEformer repository](#): 307 sensors and 16,992 five-minute observations, with the official chronological split and 12-step input/output. PEMS08 is optional confirmation.

Methods and starting code. Compact STAEformer, vanilla Transformer, GRU and STGCN/DCRNN. Replace or complement the free sensor-indexed embedding with a shared encoder using calibration summaries, graph statistics and positional encodings.

Required baselines. Original STAEformer, mean/zero/random embeddings for unseen sensors, the proposed inductive encoder and a hybrid seen-sensor model. Reproduce the adaptive-embedding benefit before changing the method.

Evaluation. Report standard forecasting and seeded 10/20/30% sensor cold-start settings. Remove held-out sensors and incident correlation edges during training, then expose only 1, 3 or 7 days of unlabelled calibration history. MAE is primary; report RMSE/MAPE by horizon and separately for seen/unseen sensors, plus at least three seeds.

Bachelor scope. Reproduce compact STAEformer on PEMS04 and its embedding ablation. Random sensor masking may be explored, but no new inductive encoder is required.

Master research question and contribution. Research question: **Can a graph-conditioned shared sensor encoder recover useful representations for newly installed sensors without harming the fixed-network model?** The contribution is the encoder, a leakage-safe cold-start protocol and analysis by degree, correlation strength and calibration length. If it fails, quantify when sensor-specific parameters are indispensable.

Practical notes. Public data and official code reduce setup work. Keep the unseen-sensor protocol separate from Project 1.2: that project learns changing edges, while this project represents nodes that were absent during training.

Project 3.2 Inductive evaluation of GRACE

Task	Self-supervised node representation learning then node classification
Recommended level	BSc or MSc
Compute	Very light; CPU or low-memory GPU
Starting paper	Zhu et al., GRACE , 2020

What you will do. GRACE makes two corrupted views of a citation graph agree. Standard evaluation is transductive: features and edges of test nodes can be visible during self-supervised pretraining. This project measures how much that access matters and develops a local variant for genuinely unseen nodes.

Project goal. Separate representation quality from the advantage of seeing the evaluation graph during pretraining.

Data. Use **Cora** and **CiteSeer**, with PubMed optional, through the [PyG Planetoid loader](#). The downstream task predicts a paper topic for each node.

Methods and starting code. Implement GRACE edge removal, feature masking, shared GCN encoder, projection head and node contrastive loss. Start from the [official GRACE repository](#). Compare Deep Graph Infomax, a feature-only MLP, GraphSAGE and a supervised GCN.

Required baselines. Random frozen encoder, GRACE frozen probe, GRACE fine-tuning and the same encoder fully supervised. Standard transductive and strict inductive pretraining must be separately labelled.

Evaluation. Accuracy and macro-F1 at 1/5/10/100% label budgets. Use at least ten seeded splits with confidence intervals. In strict induction, remove validation/test nodes and incident edges until evaluation, then use the same inference-time neighbourhood policy for all methods.

Bachelor scope. Reproduce transductive GRACE on Cora plus a second dataset, compare frozen, fine-tuned and supervised GCN, and plot label efficiency.

Master research question and contribution. Research question: **How much of GRACE's label efficiency survives strict induction, and can contrastive pretraining on sampled ego-graphs recover the loss?** Build a GraphSAGE-style local encoder and analyse performance by held-out node degree, homophily and neighbourhood size. The leakage-controlled benchmark is itself part of the contribution.

Practical notes. Small datasets make multi-seed evaluation realistic. Never place transductive and inductive figures in one undifferentiated leaderboard.

Project 3.3 Structure-aware augmentations for graph contrastive learning

Task	Self-supervised graph representation learning then graph classification
Recommended level	BSc reproduction; MSc method project
Compute	Light to moderate; CPU or low-memory GPU
Starting paper	You et al., GraphCL , NeurIPS 2020

What you will do. GraphCL learns by making two augmented versions of the same graph agree. Random deletion can destroy bridges, cycles or community structure and accidentally change the graph's meaning. This project controls that damage rather than merely tuning an augmentation rate.

Project goal. Test whether structure-aware views are more label-efficient and less sensitive to augmentation strength than uniformly random views at the same number of edits.

Data. Use **NCI1** (4,110 graphs) and **PROTEINS** (1,113) from [TU Dortmund](#) via the [PyG TUDataset loader](#). Cora/CiteSeer are optional only for a node-level spectral extension.

Methods and starting code. Start with the [official GraphCL repository](#) and a GIN encoder. Compare GraphMAE, and optionally GRACE/BGRL for node experiments. **Core MSc mechanism:** preserve combinatorial structure using bridges, articulation points, cycles or connected components. **Stretch mechanism:** protect a low-frequency Laplacian band; compare with [GCL-SPAN](#).

Required baselines. Original random GraphCL augmentations at matched edit counts, random frozen GIN, fully supervised GIN and GraphMAE. Pretrain inside each training fold for the strict protocol.

Evaluation. Accuracy and macro-F1 with confidence intervals; frozen probe and fine-tuning at 10/25/100% labels. Also report component-count change, cycle-rank change, degree-distribution distance and, for the spectral route, protected-band distance. Test edit rates such as 5/10/20/30%.

Bachelor scope. Reproduce GraphCL on NCI1, compare its four augmentations and evaluate frozen, fine-tuned and supervised GIN on identical splits. Protecting articulation points only is an optional extension.

Master research question and contribution. Research question: **At matched edit budgets, which measurable notion of structural damage best predicts downstream representation failure?** The minimum MSc contribution is one defensible structure-aware operator plus matched-budget analysis. A full combinatorial-versus-spectral comparison is an optional 30-ECTS extension. If accuracy does not improve, explain when preservation makes the two views too similar to provide contrastive signal.

Practical notes. Exact spectral work is feasible on these small graphs. Do not begin with ogbn-arxiv; scalable eigenspectrum approximation is a separate advanced problem. Label transductive test-graph exposure explicitly.

Project 3.4 Compute-aware routing between GraphGPS branches

Task	Graph regression
Recommended level	BSc reproduction; MSc preferred
Compute	Light to moderate; 8 GB GPU preferred
Starting paper	Rampášek et al., GraphGPS , NeurIPS 2022

What you will do. A GraphGPS layer computes both local message passing and global attention for every graph. This project asks whether a small gate can learn when global attention is worth its cost.

Project goal. Find an accuracy–efficiency trade-off and determine whether graphs with stronger long-range structure use the global branch more often.

Data. Use the 12,000-graph **ZINC subset** from the [PyG ZINC loader](#). Confirm only the selected final settings on **Peptides-struct** through the [PyG LRGB loader](#).

Methods and starting code. Start from the [official GraphGPS repository](#). Implement a positional/structural encoding, local GINE or GatedGCN, and global Transformer or Performer. Add a soft gate during training and an optional budgeted hard gate at inference.

Required baselines. Local-only, global-only, standard GPS and a fixed scalar mixture, followed by learned soft and hard gates. Keep depth, width and parameters approximately matched.

Evaluation. MAE with at least three seeds. Report trainable parameters, estimated FLOPs, peak memory and measured latency at a fixed batch size. Plot the accuracy–compute Pareto curve and stratify by graph size and a long-range proxy.

Bachelor scope. Implement a compact GPS layer on ZINC and reproduce local-only, global-only and combined branches with one positional encoding.

Master research question and contribution. Research question: **Can graph- or layer-conditioned routing reduce global-attention computation while retaining GraphGPS accuracy, and are gate decisions structurally interpretable?** Compare multiple compute penalties and test whether simple size/diameter/positional statistics predict routing. The contribution is the router and controlled Pareto analysis, not another GraphGPS run.

Practical notes. ZINC is light. Restrict Peptides-struct to the selected final settings. FLOPs and wall-clock latency answer different questions; report both.

Project 3.5 Question-adaptive retrieval budgets for G-Retriever

Task	Query-conditioned evidence retrieval and answer ranking
Recommended level	Advanced MSc optional; BSc retrieval-only variant
Compute	Moderate; no 7B model training required
Starting paper	He et al., G-Retriever , NeurIPS 2024

What you will do. G-Retriever scores graph elements, selects a compact connected evidence subgraph and then uses it for question answering. A single global retrieval size trades answer recall against irrelevant context. This project predicts the budget separately for each question.

Project goal. Retrieve substantially smaller evidence subgraphs without reducing answer-entity recall and ranking quality.

Data. Use [WebQSP](#), 4,737 questions, through the [PyG WebQSP loader](#) and preprocessing in the [official G-Retriever repository](#). A 1,000–2,000-question subset is acceptable for development; use the official split for final MSc reporting if feasible.

Methods and starting code. Reproduce graph-element scoring and PCST or a documented connected approximation. Compare one-hop, text-similarity top-k, personalized PageRank and fixed PCST budgets. Predict k from non-leaking query, seed-entity and relevance-score features. A language generator is optional.

Required baselines. Fixed k in {3,5,10,20}, a simple validation heuristic, non-connected top-k and an oracle budget used only for analysis. The required downstream task is answer-entity ranking, not full Llama 2–7B reproduction.

Evaluation. Answer-entity recall, Hits@1 or answer F1, retrieved node/edge count and latency. Plot the full quality–size Pareto curve, stratify by path length and question complexity, and test 5/10/20% candidate-edge deletion.

Bachelor scope. Implement retrieval and answer ranking and compare one-hop, text-only, PageRank and fixed-budget PCST. No LLM fine-tuning.

Master research question and contribution. Research question: **Can query- and retrieval-confidence features choose the smallest adequate connected subgraph per question?** Train a budget selector only on training/validation questions and analyse its calibration and failure modes. A negative result should identify which observable features predict retrieval difficulty.

Practical notes. This project has substantial preprocessing and retrieval-engineering work. It is suitable for a student who specifically wants graph RAG and is comfortable debugging data pipelines. Cache text embeddings and avoid paid APIs or 7B+ fine-tuning.

Category 4 Vision foundation models

Large self-supervised vision backbones provide general-purpose features without task-specific labels. A student project should not retrain such a backbone. It should measure what a frozen model already supplies, what a small adapter adds and whether the gain justifies its trainable parameters.

Choosing a project track

The project has two separate task tracks. EuroSAT RGB supports the Bachelor classification route. MVTec dense anomaly localisation or multispectral EuroSAT supports a more complex Master route.

Compare frozen features, an adapter and supervised training under the same split and label budget. Report trainable parameters and the evaluation protocol alongside accuracy.

Learn first and find data

- Core reading: [DINO](#), [DINOv2](#), the [DINOv2 model card and code](#), [DINOv3](#) and [AnomalyDINO](#).
- Data and model hubs: [MVTec AD](#), [EuroSAT](#), [Hugging Face models](#) and [timm](#). Always read model and dataset licences before an industry-facing extension.
- Stanford CS224W does not cover vision foundation models directly. Use its project rubric for scoping and reproducibility, but use the official DINO papers and repositories for technical guidance.

Project 4.1 Adapting DINO backbones for land cover or industrial anomalies

Task	Image classification or image/pixel anomaly detection
Recommended level	BSc or MSc with separate tracks
Compute	Moderate; frozen ViT-S practical on 8 GB, dense MSc work may need more
Starting paper	Oquab et al., DINOv2 , 2023

What you will do. Use a pretrained self-supervised vision transformer as a frozen feature extractor. Train a small head or adapter for a specialised task. Measure the performance of frozen features and test whether an adapter still contributes when a newer backbone is used.

Project goal. Quantify the trade-off among task performance, number of trainable parameters and supervision. Choose **one** track: EuroSAT RGB land-cover classification or MVTec industrial anomaly detection/localisation.

Data. BSc/default classification track: [EuroSAT](#), 27,000 Sentinel-2 patches in 10 classes. Use the **RGB** version for ordinary DINO backbones; the 13-band version requires an explicit multispectral adapter and is a separate MSc question. **MSc anomaly track:** [MVTec AD](#), more than 5,000 images in 15 categories with pixel masks, CC BY-NC-SA 4.0 for non-commercial research.

Methods and starting code. Default to DINOv2 ViT-S/14 from the [official Apache-2.0 repository and model card](#). Compare frozen linear/MLP probes, k-NN or [AnomalyDINO](#) patch similarity, and a lightweight multi-scale adapter. DINOv3 is optional through the [official repository](#); its weights are gated under a custom Meta licence.

Required baselines. From-scratch CNN, ImageNet-pretrained ResNet or ViT, frozen DINOv2 with no adapter, lightweight adapter and full fine-tuning only if hardware permits. On MVTec, include a simple nearest-neighbour patch baseline and keep zero-shot, one-shot, few-shot and full-shot protocols separate.

Evaluation. EuroSAT: top-1 accuracy, macro-F1, label-efficiency curve and trainable parameters. MVTec: image AUROC plus pixel AUROC/PRO or pixel F1-max, reported by category and protocol. Measure inference memory/latency. Never compare a one-shot image metric directly with a zero-shot pixel metric.

Bachelor scope. Use EuroSAT RGB. Compare a frozen DINOv2 ViT-S linear probe with an ImageNet-pretrained CNN and, if feasible, a small adapter at several label budgets.

Master research question and contribution. Choose one question. **Dense anomaly route:** can a supervised multi-scale adapter over frozen patch tokens improve few-shot localisation at a fixed label and parameter budget, and does the gain remain after switching DINOv2 to DINOv3? **Remote-sensing route:** can a small multispectral input adapter exploit 13 bands without full backbone fine-tuning? Report cross-version or cross-band ablations so the adapter's value is isolated.

Practical notes. Do not attempt DINOv3 ViT-7B. DINOv2 is ungated and Apache 2.0; build the pipeline on it first. DINOv3's licence is non-standard and gated but not stated as categorically non-commercial; an industry use requires institutional/legal review. MVTec itself is non-commercial, so it cannot serve as an industry dataset without separate permission.

Selection guidance

Suggested starting points

BSc with lower setup risk	1.1 C-MAPSS, 1.4 MovieLens, 2.1 UCI-HAR, 3.2 GRACE reproduction, 4.1 EuroSAT RGB
MSc with a defined method extension	3.1 inductive STAEformer, 3.3 structure-aware GraphCL, 1.2 dynamic traffic graph, 3.4 GraphGPS routing
MSc with higher methodological risk	2.2 lead-aware latent prediction, 2.3 decomposition-aware forecasting, 2.5 physics-grounded vibration graphs
Preliminary checks required	2.4 HEAPO after a licence and data check; 3.5 G-Retriever for students prepared for substantial retrieval engineering

Week-one feasibility checklist

- Download the exact dataset and record its version, size, licence and checksum or release date.
- Run the simplest baseline from raw input to final metric on a small subset.
- Write the independent split unit and verify there is no subject/patient/household/engine/bearing overlap.
- Record the anchor paper, official repository commit or release, and any preprocessing deviations.
- Estimate training time and peak memory before planning seeds or ablations.
- For MSc work, write the research question, proposed mechanism, main alternative explanation and minimum viable ablation table.

Recurring failure modes

- Comparing with a number produced by a different split, metric, horizon, shot count or evaluation level.
- Using windows from the same physical unit in both training and test sets.
- Calling an authors' repository run a research contribution.
- Reporting only in-distribution performance when transfer or cold start is the stated motivation.
- Omitting the random frozen encoder in a self-supervised study.
- Treating a recent preprint figure as a settled benchmark.
- Expanding to a second dataset before the first baseline and evaluation protocol are stable.

Select one primary question, fix the evaluation protocol before extensive experimentation and report negative as well as positive results. Two industrial-partner projects with Tetra Pak and Siemens Gamesa are offered separately and remain outside this portfolio.